

Тема 02

Стандартизация кода и версионирование данных

Стандартизация ML-кода и документация

Общие правила делают ML-код понятным команде

Уровень	О чём договариваемся	Пример
Оформление	Единая запись	PEP 8: имена, отступы
Структура	Границы ответственности	Расчёт отдельно от I/O
Контракт	Вход, выход, ограничения	Один ответ на каждый вход

Сегодня — код и документация. Далее — хранение и версии данных.

Разные проверки обнаруживают разные ошибки

Проверка	Что проверяет	Пример
Форматтер	Оформление	Отступы
Линтер	Известные конструкции	Неопределённое имя
Проверка типов	Согласованность типов	str вместо float
Тесты	Поведение на примерах	Границы [0, 1]
Ревью	Смысл решения	Утечка target

Ruff не заменяет тесты, проверку типов и ревью.

Явные зависимости упрощают изменение кода

Зависимость от внешнего состояния

```
def score():  
    return rmse(  
        y_valid, model.predict(x_valid)  
    )
```

Входы видны в сигнатуре



```
def score(y_true, y_pred):  
    return rmse(y_true, y_pred)
```

DRY: единое правило расчёта

KISS: простая функция

YAGNI: без лишних абстракций

Связанные вычисления — вместе; обучение и запись файлов — отдельно.

Форматтер меняет оформление, а не назначение кода

До форматирования

```
config={"batch_size":32,  
"shuffle":True,}
```

После ruff format



```
config = {  
    "batch_size": 32,  
    "shuffle": True,  
}
```

```
ruff format src tests
```

```
ruff format --check src tests
```

Назначение имён, границы функций и корректность метрик остаются за нами.

Линтер находит проблемы, но не всегда знает исправление

```
import json
```

F401

Неиспользуемый импорт

```
def predict_batch(model, features):  
    return model.predict(feature)
```

F821

Неопределённое имя feature

```
def add_metric(value, history=[]):  
    history.append(value)  
    return history
```

B006

Общий список между вызовами

--fix применяет доступные безопасные исправления. После — читаем diff.

Конфигурация закрепляет общие правила команды

```
[tool.ruff]
line-length = 88
target-version = "py313"
```

```
[tool.ruff.lint]
select = [
    "E4", "E7", "E9", "F", "I", "B",
]
```

E4, E7, E9 · F

Базовые ошибки и имена

I · B

Импорты и опасные конструкции

Версия Ruff + конфигурация

Одинаковые проверки в команде

target-version задаёт допустимый синтаксис; Python эта настройка не устанавливает.

pip, conda и uv управляют разными частями окружения

Инструмент	Что устанавливает	Управление окружением
pip	Python-пакеты	venv создаём отдельно
conda	Python и нативные библиотеки	Env и каналы conda
uv	Python и Python-пакеты	Проект: .venv + uv.lock

Выбираем инструмент под зависимости проекта, а не только под скорость установки.

Одна установка NumPy — три отдельных сценария

pip + venv

```
python -m venv .venv  
source .venv/bin/activate  
python -m pip install numpy
```

conda

```
conda create -n demo -c conda-forge python=3.13 numpy  
conda activate demo  
python -c "import numpy"
```

uv

```
uv init --python 3.13  
uv add numpy  
uv run python -c "import numpy"
```

Это альтернативы. Для нового uv-проекта берём отдельную пустую папку.

Окружение и фиксация зависимостей решают разные задачи

Артефакт	Что хранит	Что важно помнить
pyproject.toml	Требования и настройки	Не является lock-файлом
requirements.txt	Список для установки pip	Версии могут быть не точными
uv.lock	Версии всех зависимостей	Храним в Git
.venv/	Установленный env	Пересоздаём; не коммитим

uv sync --locked: установить env по lock-файлу; остановиться, если его нужно изменить.

Тесты помогают сохранить результат при рефакторинге

Скрипт оценки модели

Предсказания + метрика + запись



Отдельная функция метрики

`rmse(y_true, y_pred)`
→ float

Ручной пример

`y_true = [0, 0]`

`y_pred = [0, 4]`

$$\text{RMSE} = \sqrt{((0^2 + 4^2) / 2)} = \sqrt{8}$$

MSE = 8 — этот тест различает две формулы

Ручной пример проверяет формулу; прежние тесты — интеграцию и формат отчёта.

Тип DataFrame не гарантирует корректность данных

```
def validate_predictions(  
    frame: pd.DataFrame,  
) -> None:
```

Тип не задаёт

Колонки

Единицы измерения

Допустимые значения

Контракт: probability — конечное число от 0 до 1

-0.1

ошибка

0.0

допустимо

1.0

допустимо

1.1

ошибка

NaN и $\pm\infty$:

ошибка

Аннотация описывает тип; валидация и тесты проверяют требования к данным.

Docstring объясняет условия использования функции

```
"""Вычислить RMSE.
```

Args:

 y_true: Истинные конечные значения.

 y_pred: Предсказания в том же порядке.

Returns:

 float в единицах целевой величины.

Raises:

 ValueError: Пустые массивы, разная
 длина или неконечные значения.

```
"""
```

Args

Какие данные нужны

Returns

Результат и ограничения

Raises

Ошибки использования

PEP 257 — общие соглашения. Здесь используем Google-style; на слайде фрагмент.

Google-style и NumPy-style описывают один контракт

Google-style · фрагмент

Args:

seconds (float): Время в секундах.

Returns:

float: Время в минутах.

NumPy-style · фрагмент

Parameters

seconds : float

Время в секундах.

Returns

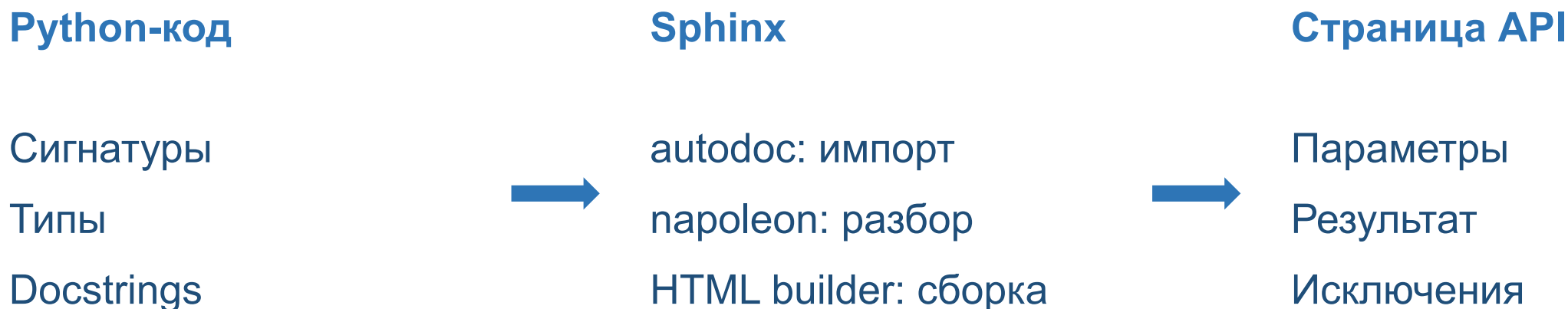
float

Время в минутах.

Одна функция: to_minutes(seconds: float) -> float

Меняется запись, не поведение. Sphinx + napoleon понимают оба стиля.

Sphinx собирает документацию API из кода



```
python -m sphinx -b html -W docs docs/_build/html
```

-W: предупреждения делают сборку неуспешной

Описание API берём из кода; сценарии использования пишем отдельно.

Hooks дают локальную проверку, CI — общую

Локально

git add → hooks → commit

Если hook исправил файл:

git diff → git add → повторный commit

На сервере

PR → CI

Стиль

Тесты

Документация

Hooks можно обойти. Для общей гарантии нужны обязательные проверки на PR.

Качество кода требует нескольких уровней проверки

Уровень	Средство	Что остаётся проверить
Единый стиль	Форматтер и линтер	Смысл и архитектура
Окружение	Менеджер + lock-файл	Python, ОС и данные
Поведение	Типы и тесты	Полнота сценариев
Контракт	Docstrings и Sphinx	Соответствие реализации
Процесс	Hooks и CI	Решения на ревью

Автоматизация проверяет правила; разработчик отвечает за их достаточность.